

Neural Networks - Multilayer Perceptrons

Enrique Romero

Computational Intelligence
Master in Artificial Intelligence

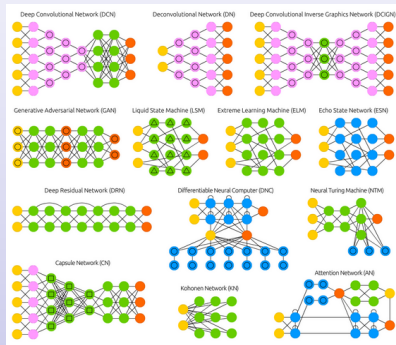
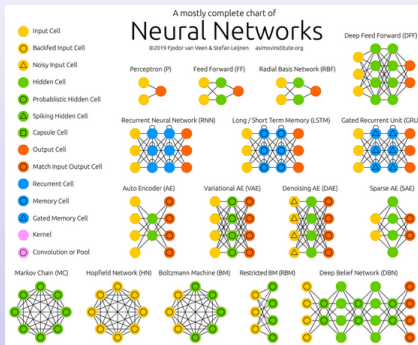
Soft Computing Group
Computer Science Department
Universitat Politècnica de Catalunya, Barcelona, Spain

- 1 Introduction
- 2 Linear Regression
- 3 General Machine Learning Issues
- 4 Single-layer Perceptrons
- 5 Multilayer Perceptrons

- 1 Introduction
 - The Neural Network Zoo
 - Definition
 - Biological Motivation
 - A Bit of History
- 2 Linear Regression
- 3 General Machine Learning Issues
- 4 Single-layer Perceptrons
- 5 Multilayer Perceptrons

- 1 Introduction
 - The Neural Network Zoo
 - Definition
 - Biological Motivation
 - A Bit of History
- 2 Linear Regression
- 3 General Machine Learning Issues
- 4 Single-layer Perceptrons
- 5 Multilayer Perceptrons

<https://www.asimovinstitute.org/neural-network-zoo>



- 1 Introduction
 - The Neural Network Zoo
 - **Definition**
 - Biological Motivation
 - A Bit of History
- 2 Linear Regression
- 3 General Machine Learning Issues
- 4 Single-layer Perceptrons
- 5 Multilayer Perceptrons

Neural Networks are defined by:

- A set of interconnected **units** (neurons)
- A set of **connections** (synapses or weights) among the units

Types of Neural Networks:

- **Feed-forward Neural Networks**, including
 - Multilayer Perceptrons
 - Radial-basis Function Networks
 - Convolutional Neural Networks
 - ...
- **Recurrent Neural Networks**, including
 - Hopfield Networks
 - Boltzmann and Restricted Boltzmann Machines
 - Long Short-Term Memory Neural Networks
 - ...

In these slides we will focus on Multilayer Perceptrons

Feed-forward Neural Networks are neural systems defined by:

- A set of interconnected **units** (neurons)
- A set of **directed connections** (synapses or weights) among the units **without loops**

Each unit

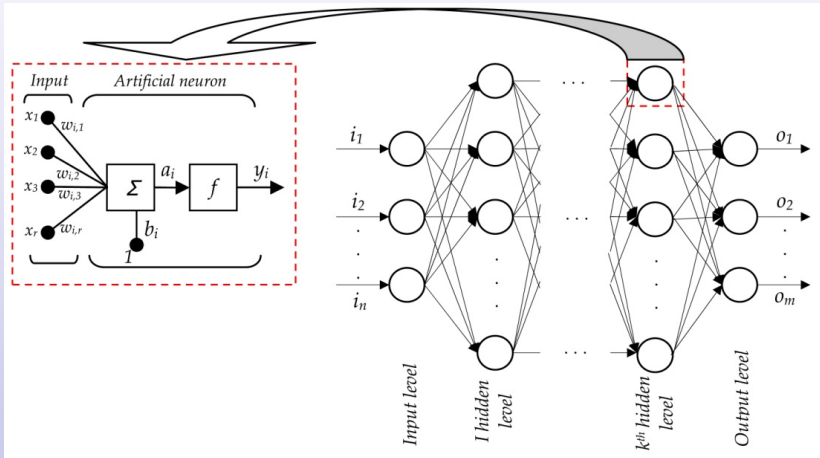
- Receives information of the units with connections **to** this unit
- Sends information to the units with connections **from** this unit

Typically,

- There are a number of **input units**, where the process starts
- Some of the units are **output units**, with the network results
- The rest of units are called **hidden** units
- Units are usually structured by **layers**

A Typical Example

Typical architecture of a Feed-forward Neural Network



Different Types of Computations

The computation made by every unit depends on the type of neural networks we are working with, and usually consists in two steps:

- An **aggregation function**, that combines the information received by the unit with the weights (converting multidimensional information into a scalar)
- An **activation function**, that transforms (usually in a non-linear way) the output of the aggregation function

The output of the aggregation function is named **net-input** or **pre-activation**

The output of the activation function (the output of the unit) is named the **net-output** or **activation** of the unit

Input units are fed with the **data** (one example at a time)

Different Types of Computations

Examples of aggregation functions:

- For Multilayer Perceptrons (MLPs), the inner product between the weights and the activation of the previous units:

$$\text{net}_i = \omega_i \cdot \text{act_to}_i = \sum_j \omega_{ij} \text{act_to}_{ij}$$

- For Radial-basis Function Networks, the squared Euclidean distance between the weights and the activation of the previous units:

$$\text{net}_i = ||\omega_i - \text{act_to}_i||^2 = \sum_j (\omega_{ij} - \text{act_to}_{ij})^2$$

- For Convolutional Neural Networks, the convolution operation

Although in this lecture we will focus on MLPs, similar ideas apply for other types of neural networks

Different Types of Computations

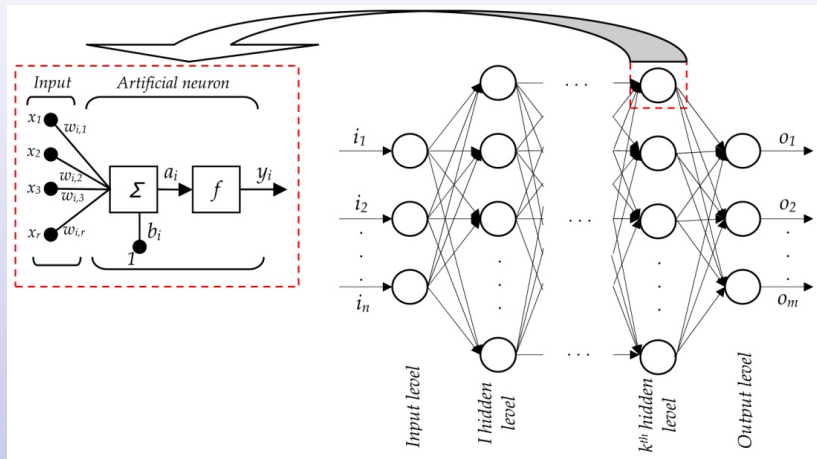
Examples of activation functions:

- Identity function: $f(z) = z$
- Logistic function: $f(z) = \frac{1}{1+e^{-z}}$
- Hyperbolic tangent function: $f(z) = \frac{e^z - e^{-z}}{e^z + e^{-z}}$
- Gaussian function: $f(z) = e^{-z/\sigma}$
- Rectifier linear function: $f(z) = \max(0, z)$
- Softplus function: $f(z) = \log(1 + e^z)$
- Softmax function (output units): $f(z)_c = \frac{e^{z_c}}{\sum_k e^{z_k}}$
- ... and many more

In many cases, the aggregation and the activation function are considered a single function, since every activation function is usually combined with a unique aggregation function

A Typical Example (revisited)

How is the computation performed?



Two Problems Related to Feed-forward Neural Networks

There are two main problems related to Feed-forward Neural Networks (and most Neural Networks):

- **(easy)** Output Computation: given
 - An architecture (including number of hidden layers, connections, aggregation functions and activations functions)
 - A set of weights θ
 - An input vector $\mathbf{x}$

compute the output value of the network $h_{\theta}(\mathbf{x})$

- **(hard)** Training / Learning: given
 - A data set $\mathbf{D} = \{(\mathbf{x}^{(1)}, y^{(1)}), \dots, (\mathbf{x}^{(N)}, y^{(N)})\}$
 - A cost function $J(\theta, \mathbf{D})$
 - An architecture (usually set a priori)

find a set of weights θ that optimizes the cost function

1 Introduction

- The Neural Network Zoo
- Definition
- **Biological Motivation**
- A Bit of History

2 Linear Regression

3 General Machine Learning Issues

4 Single-layer Perceptrons

5 Multilayer Perceptrons

The term “Artificial Neural Network” was originally motivated by the idea of modeling networks of real neurons in the brain

Nowadays, we know that the artificial neurons used to construct artificial neural networks are very primitive approximations to those found in the brain

Therefore, also the neural network architectures used are extremely simple approximations of the brain structure

However, the remarkable progress made during the past two decades inspired by this neurobiological analogy is really impressive

Biological neurons (a very simplified version):

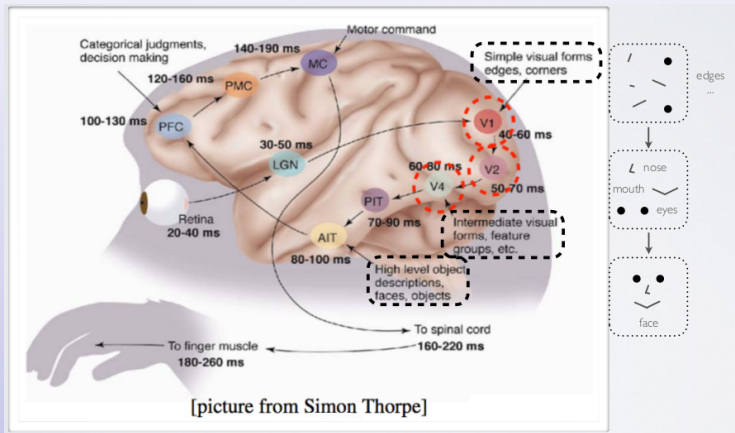
- Receive information from other neurons through the **dendrites**
- Process the information in their cell body (**soma**)
- Send information through a “cable” called an **axon**
- The point of connection between the axon branches and the dendrites of other neurons are called **synapses**
- The estimated number of synapses in the brain is 10^{15}

The “information” generated has the form of a spike in the electric potential (voltage) of the axon

Depending on the particular connections of the neurons, the information sent by one neuron can **excite** or **inhibit** other neurons

Biological Layers

The visual cortex of the brain processes the information by layers:
V1, V2, V4, PIT, AIT,...



Many Differences between the Brain and Neural Networks

- In the brain there are many types of different neurons (unipolar, bipolar, multipolar, glutamatergic, dopaminergic, serotonergic,...)
- The brain is specialized into many different areas (cortices), responsible for particular tasks: vision, hearing, speech, touch,... with inter-regional circuits among these areas
- Additional association areas link information affecting multiple sensations, to produce a meaningful perceptual experience of the world, enable us to interact effectively, and support abstract thinking and language
- Although biological neurons are several orders of magnitude slower than silicon logic gates (10^{-3} vs. 10^{-9} seconds), it is efficient for many tasks (by using massive interconnections between the neurons)

1 Introduction

- The Neural Network Zoo
- Definition
- Biological Motivation
- A Bit of History

2 Linear Regression

3 General Machine Learning Issues

4 Single-layer Perceptrons

5 Multilayer Perceptrons

A Bit of History

1943	Logical calculus model of McCulloch & Pitts [McCulloch and Pitts, 1943]
1949	Hebb's learning for self-organizing synaptic modification [Hebb, 1949]
1958	Perceptron supervised learning model [Rosenblatt, 1958]
1960	Adaline (ADaptive LINear Element) model based on Least Mean-Squares and the delta rule [Widrow and Hoff, 1960]
1967	First Multilayer networks [Amari, 1967, Ivakhnenko, 1971]
1967	Minsky extends the results of McCulloch and Pitts and puts them in the context of the theory of computation [Minsky, 1967]
1969	Minsky & Papert's book <i>Perceptrons</i> [Minsky and Papert, 1969]
1970s	Self-Organizing models using competitive learning and associative memories [Anderson, 1972, Kohonen, 1972, Grossberg, 1972]
1980	Adaptive Resonance Theory models, started by [Grossberg, 1980]
1982	Hopfield Networks, energy-based models [Amari, 1972, Hopfield, 1982]
1985	Boltzmann Machines [Ackley et al., 1985] and Restricted Boltzmann Machines [Smolensky, 1986]

In 1969, [Minsky and Papert, 1969] formally proved that perceptron-like networks had strong limitations, and (wrongly) conjectured that these limitations would be shared by networks with multiple layers

This conjecture, together with the lack of an efficient algorithm to train multilayer networks, lead to what is called the “first neural network winter”

Anyway, there was still good research in the 1970s and early 1980s

A Bit of History

1986	Backpropagation algorithm for Multilayer Perceptrons [Werbos, 1974, Rumelhart et al., 1986b] and PDP books [Rumelhart and McClelland, 1986]
1986	Auto-encoders [Rumelhart et al., 1986a]
1988	Radial-basis Function Networks [Broomhead and Lowe, 1988, Poggio and Girosi, 1990]
1988-1990	Time-Delay Neural Networks [Lang et al., 1990]
1989	Convolutional Neural Networks [LeCun et al., 1989]
1992	Support Vector Machines [Boser et al., 1992, Cortes and Vapnik, 1995]
1997	Long Short-Term Memory [Hochreiter and Schmidhuber, 1997]
1999	Contrastive Divergence [Hinton, 1999, Hinton, 2002]
2006	Deep Belief Networks [Hinton et al., 2006]
2006	Deep Neural Networks and Deep Auto-encoders [Hinton and Salakhutdinov, 2006]
2012	Large Scale Visual Recognition Challenge 2012 winner [Krizhevsky et al., 2012]

The end of the first neural network winter (1986) was mainly due to scientific reasons:

- Use of differential calculus:
 - Replace non-differentiable activation functions (step function) by differentiable ones
 - Introduce differentiable cost functions

With these changes, training a multilayer network can be seen as the minimization of a differentiable function wrt the parameters

- Application of gradient-based optimization methods
- Efficient computation of the derivatives involved in the optimization process (backpropagation)
- Theoretical results supporting the universality of these models

After the initial optimism, it was observed that networks with many layers:

- Were difficult to train with backpropagation
- Did not use the power they were supposed to have (only the weights in the final layers were useful)

As a consequence, results were not as good as expected, and there were very few successful applications of networks with more than two layers (with the exception of Convolutional Neural Networks)

Therefore, researchers began to explore other alternatives, such as Support Vector Machines, kernel methods in general, etc, leading to the “second neural network winter”

Anyway, there was still good research in the 1990s and early 2000s

The end of the second neural network winter (2012) was mainly due to technological and computational reasons:

- Availability of large quantities of data
- Computers were millions of times faster
- GPUs allowed massive computations in parallel
- Development of automatic differentiation methods, that allow rapid experimentation of different architectures and combinations of them

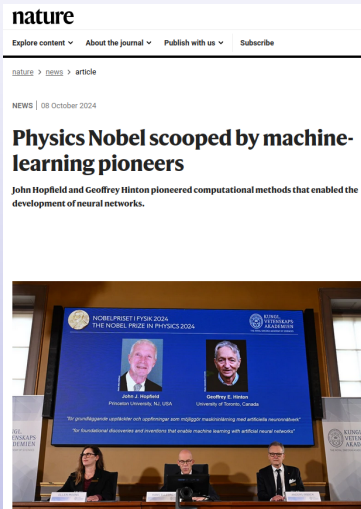
together with scientific ones:

- Development of faster gradient-based optimization methods
- Development of clever architectures

In this context, it is possible to train very large (typically deep) multilayer networks, where (as experience shows) backpropagation works well

Since 2012 a lot has happened in this field, we currently are in a “neural network spring” that we call **Deep Learning**...

...In 2024 J. Hopfield and G. Hinton receive Nobel Prize in Physics



But... can anyone be sure there will not be a “third neural network winter”?

- 1 Introduction
- 2 Linear Regression
 - Normal Equations
 - Gradient Descent
- 3 General Machine Learning Issues
- 4 Single-layer Perceptrons
- 5 Multilayer Perceptrons

WARNING: We are aware that the Linear Regression model

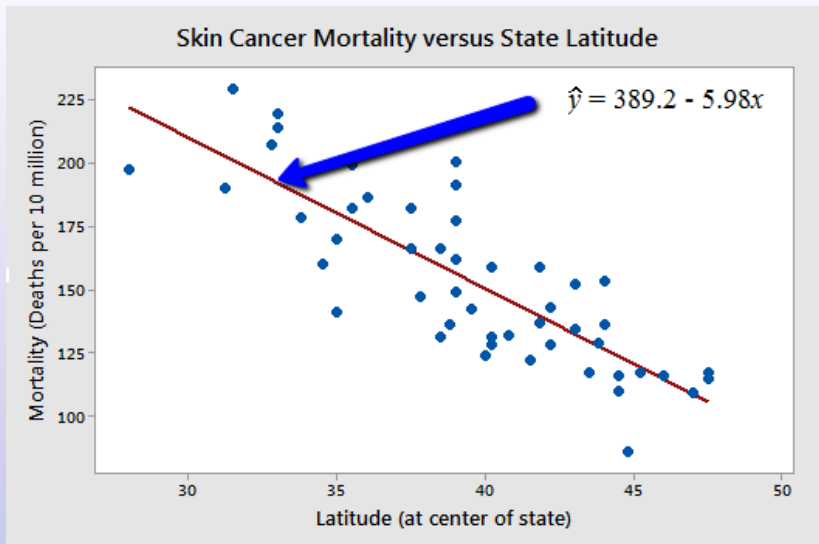
- Probably is well-known by most of you
- Is already explained (in more or less detail) in other courses (**Introduction to Machine Learning**, for example)

However, it is worth explaining it here because

- It contains the main issues of a Machine Learning problem
- It can be seen as a very simple example of Neural Network (probably the simplest one)
- It can be solved in two different ways, being a very nice introduction to one of the main problems in Neural Networks

If you think we can skip any explanation, please feel free to say it
Anyway, we will not spend too much time...

Linear Regression: Statement of the Problem



Linear Regression: Statement of the Problem

Type of problem: Regression

- Given a data set with N examples
 $D = \{(\mathbf{x}^{(1)}, y^{(1)}), \dots, (\mathbf{x}^{(N)}, y^{(N)})\}$
- We want to model the relationship between the **scalar** variable y (the **target**) and the d -dimensional **vector** $\mathbf{x} = (\mathbf{x}_1, \dots, \mathbf{x}_d)$ of explanatory variables or features (the **input**)

Model representation: linear

- $h_{\theta}(\mathbf{x}) = \theta_0 + \theta' \cdot \mathbf{x}$
- $\theta = (\theta_1, \dots, \theta_d)'$ is the **vector of parameters**
- θ_0 is a **bias term**, that can be seen as a standard parameter if we add a constant 1 to the input $\mathbf{x} = (1, \mathbf{x}_1, \dots, \mathbf{x}_d)$

Note that y is a scalar and $\mathbf{x}$ is a vector

Linear Regression: Statement of the Problem

Therefore, we would like that (if possible)

$$y^{(i)} = h_{\boldsymbol{\theta}}(\mathbf{x}^{(i)}) = \boldsymbol{\theta}' \cdot \mathbf{x}^{(i)} \quad i = 1, \dots, N$$

To that end, we define a sum-of-squares **cost function**

$$J(\boldsymbol{\theta}, \mathbf{D}) = \frac{1}{2} \sum_{i=1}^N \left(y^{(i)} - h_{\boldsymbol{\theta}}(\mathbf{x}^{(i)}) \right)^2 = \frac{1}{2} \sum_{i=1}^N \left(y^{(i)} - \boldsymbol{\theta}' \cdot \mathbf{x}^{(i)} \right)^2$$

that we want to **minimize** [Legendre, 1805, Gauss, 1809]

- 1 Introduction
- 2 Linear Regression
 - Normal Equations
 - Gradient Descent
- 3 General Machine Learning Issues
- 4 Single-layer Perceptrons
- 5 Multilayer Perceptrons

Linear Regression: A First Solution (Normal Equations)

In this particular case, and given the linear nature of the problem, it can be proved that

$$J(\theta, \mathbf{D}) = \frac{1}{2}(\mathbf{Y} - h_{\theta}(\mathbf{X}))'(\mathbf{Y} - h_{\theta}(\mathbf{X})) = \frac{1}{2}(\theta' \mathbf{X}' \mathbf{X} \theta - 2 \theta' \mathbf{X}' \mathbf{Y} + \mathbf{Y}' \mathbf{Y})$$

where

- $\mathbf{Y} = (y^{(1)}, \dots, y^{(N)})'$ is the vector of target values (size $N \times 1$)
- $\mathbf{X}_{ij} = x_j^{(i)}$ is the matrix of inputs (size $N \times d$)
- $h_{\theta}(\mathbf{X}) = \mathbf{X} \theta$ is the output for the whole data set

Computing the gradient of $J(\theta, \mathbf{D})$ wrt θ and setting it equal to 0 we have

$$\frac{\partial J(\theta, \mathbf{D})}{\partial \theta} = \mathbf{X}' \mathbf{X} \theta - \mathbf{X}' \mathbf{Y} = 0$$

Linear Regression: A First Solution (Normal Equations)

Therefore (**normal equations**)

$$\mathbf{X}' \mathbf{X} \boldsymbol{\theta} = \mathbf{X}' \mathbf{Y}$$

leading to

$$\boldsymbol{\theta} = (\mathbf{X}' \mathbf{X})^{-1} \mathbf{X}' \mathbf{Y}$$

The $d \times d$ matrix $\mathbf{X}' \mathbf{X}$ is the **autocorrelation matrix**, closely related to the covariance matrix

The $d \times N$ matrix $(\mathbf{X}' \mathbf{X})^{-1} \mathbf{X}'$ is a particular case of the the **Moore-Penrose pseudoinverse**, that generalizes the concept of inverse matrix

Even when $(\mathbf{X}' \mathbf{X})^{-1}$ is singular, the normal equations can be solved, and therefore **the solution of the linear regression problem can always be computed**

- 1 Introduction
- 2 Linear Regression
 - Normal Equations
 - Gradient Descent
- 3 General Machine Learning Issues
- 4 Single-layer Perceptrons
- 5 Multilayer Perceptrons

Linear Regression: A Second Solution (Gradient Descent)

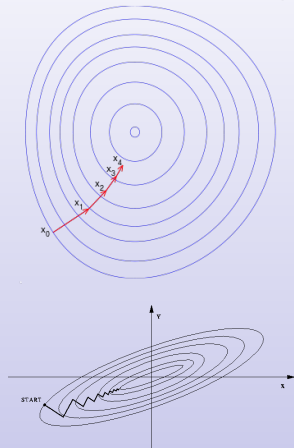
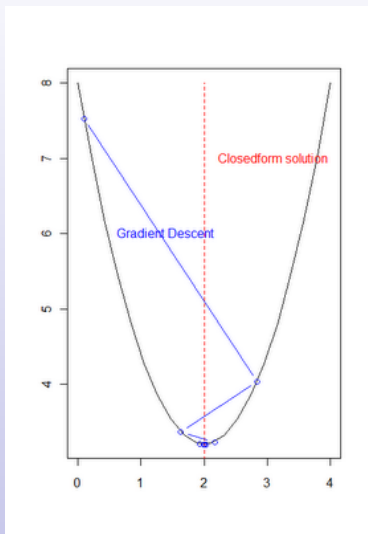
An alternative solution to the normal equations (that can be used in more general frameworks) is to perform **gradient descent in the space of the parameters**

A general description of gradient descent:

- It is a **first-order iterative optimization algorithm** for finding the minimum of a function
- It takes steps proportional to the **negative gradient** of the function (or an approximation) at the current point
- It usually obtains a **local minimum** of the function

To my knowledge, the gradient descent algorithm (also named **delta rule**) was suggested by Cauchy (1847), originally described in [Hadamard, 1908], and first used in Neural Networks by [Widrow and Hoff, 1960]

Linear Regression: A Second Solution (Gradient Descent)



Linear Regression: A Second Solution (Gradient Descent)

A general algorithm for gradient descent:

Input: (data set $\mathbf{D}$, learning rate $\lambda > 0$)

Initialize θ_0

$t = 0$

repeat until convergence:

 Compute the gradient $g_t = \partial J(\theta_t, \mathbf{D}) / \partial \theta$

$\theta_{t+1} = \theta_t - \lambda g_t$

$t = t + 1$

Return θ_t

Linear Regression: A Second Solution (Gradient Descent)

In the case of linear regression,

$$\frac{\partial J(\boldsymbol{\theta}_t, \mathbf{D})}{\partial \boldsymbol{\theta}} = \mathbf{X}' \mathbf{X} \boldsymbol{\theta}_t - \mathbf{X}' \mathbf{Y} = \mathbf{X}' \left(h_{\boldsymbol{\theta}_t}(\mathbf{X}) - \mathbf{Y} \right)$$

Since the linear regression problem leads to a convex surface, **the gradient descent algorithm converges to its global minimum** [Horst and Tuy, 1993]

Linear Regression: Comparison of Solutions

Advantages of the normal equations:

- The solution is obtained in one step, without iterations
- There are no parameters to select (learning rate)

Advantages of the gradient descent algorithm:

- It does not present numerical problems related to the inverse of $(\mathbf{X}' \mathbf{X})^{-1}$
- It may be faster when the number of features d is large, since the computation cost of $(\mathbf{X}' \mathbf{X})^{-1}$ is $O(d^3)$
- **It is a general framework that can be used in other problems**

- 1 Introduction
- 2 Linear Regression
- 3 General Machine Learning Issues**
 - Type of Problem, Model Representation, Cost Function and Optimization Technique
 - Examples
- 4 Single-layer Perceptrons
- 5 Multilayer Perceptrons

- 1 Introduction
- 2 Linear Regression
- 3 General Machine Learning Issues
 - Type of Problem, Model Representation, Cost Function and Optimization Technique
 - Examples
- 4 Single-layer Perceptrons
- 5 Multilayer Perceptrons

Type of Problem, Model Representation, Cost Function and Optimization Technique

The previously explained example of linear regression contains all the basic ingredients needed in a problem that is going to be tackled with Neural Networks:

- Type of Problem
- Model Representation
- Cost Function
- Optimization Technique

Not specific for Neural Networks, but for Machine Learning models in general

Type of Problem, Model Representation, Cost Function and Optimization Technique

Suppose that we are given a data set with N examples

$\mathbf{D} = \{(\mathbf{x}^{(1)}, y^{(1)}), \dots, (\mathbf{x}^{(N)}, y^{(N)})\}$ from which we want to learn something (this is the aim of **Machine Learning**)

The **Type of Problem** is the nature of the problem we are trying to solve: regression, classification, clustering, etc

- In the example of linear regression, it is regression

The **Model Representation** is the form of the output function (hypothesis) of the model

- In the example of linear regression, it is a linear function that depends on several parameters $h_{\theta}(\mathbf{x}) = \theta_0 + \theta' \cdot \mathbf{x}$

Type of Problem, Model Representation, Cost Function and Optimization Technique

The **Cost Function** is the (usually scalar) function to be optimized, which **strongly depends on the type of problem** (typically based on the **empirical risk minimization**) and is a **function of the data and the parameters**

- In the example of linear regression, it is the sum-of-squares error function $J(\theta, \mathbf{D}) = \frac{1}{2} \sum_{i=1}^N (y^{(i)} - h_{\theta}(\mathbf{x}^{(i)}))^2$

The **Optimization Technique** is used to optimize the cost function

- In the example of linear regression, it can be done with the normal equations or gradient descent

- 1 Introduction
- 2 Linear Regression
- 3 General Machine Learning Issues**
 - Type of Problem, Model Representation, Cost Function and Optimization Technique
 - **Examples**
- 4 Single-layer Perceptrons
- 5 Multilayer Perceptrons

Linear Regression has already been explained:

- **Type of problem:** regression
- **Model representation:**

$$h_{\theta}(\mathbf{x}) = \theta_0 + \theta' \cdot \mathbf{x}$$

- **Cost function:** sum-of-squares error

$$J(\theta, \mathbf{D}) = \frac{1}{2} \sum_{i=1}^N \left(y^{(i)} - h_{\theta}(\mathbf{x}^{(i)}) \right)^2$$

- **Optimization technique:** normal equations, gradient descent

Ridge Regression is a regularized version of linear regression, that can be represented as:

- **Type of problem:** regression
- **Model representation:** the same as linear regression

$$h_{\theta}(\mathbf{x}) = \theta_0 + \theta' \cdot \mathbf{x}$$

- **Cost function:** (2-norm) regularized sum-of-squares error

$$J(\theta, D) = \frac{1}{2} \sum_{i=1}^N \left(y^{(i)} - h_{\theta}(\mathbf{x}^{(i)}) \right)^2 + \alpha \sum_{j=0}^d \theta_j^2$$

- **Optimization technique:** modified normal equations, gradient descent

By the way, it can be proved that minimizing the cost function of ridge regression

$$J(\boldsymbol{\theta}, \mathbf{D}) = \frac{1}{2} \sum_{i=1}^N \left(y^{(i)} - h_{\boldsymbol{\theta}}(\mathbf{x}^{(i)}) \right)^2 + \alpha \sum_{j=0}^d \theta_j^2$$

is equivalent to solving the following constrained optimization problem:

$$\begin{array}{ll} \text{Minimize } \boldsymbol{\theta} & \frac{1}{2} \sum_{i=1}^N \left(y^{(i)} - h_{\boldsymbol{\theta}}(\mathbf{x}^{(i)}) \right)^2 \\ \text{subject to} & \sum_{j=0}^d \theta_j^2 \leq T \end{array}$$

which is a form of **structural risk minimization**

Lasso Regression is another regularized version of linear regression, that can be represented as:

- **Type of problem:** regression
- **Model representation:** the same as linear regression

$$h_{\theta}(\mathbf{x}) = \theta_0 + \theta' \cdot \mathbf{x}$$

- **Cost function:** (1-norm) regularized sum-of-squares error

$$J(\theta, D) = \frac{1}{2} \sum_{i=1}^N \left(y^{(i)} - h_{\theta}(\mathbf{x}^{(i)}) \right)^2 + \alpha \sum_{j=0}^d |\theta_j|$$

- **Optimization technique:** quadratic programming techniques, least angle regression for Lasso

Similar to ridge regression, it can be proved that minimizing the cost function of lasso regression

$$J(\boldsymbol{\theta}, \mathbf{D}) = \frac{1}{2} \sum_{i=1}^N \left(y^{(i)} - h_{\boldsymbol{\theta}}(\mathbf{x}^{(i)}) \right)^2 + \alpha \sum_{j=0}^d |\theta_j|$$

is equivalent to solving the following constrained optimization problem:

$$\begin{aligned} &\text{Minimize } \boldsymbol{\theta} \quad \frac{1}{2} \sum_{i=1}^N \left(y^{(i)} - h_{\boldsymbol{\theta}}(\mathbf{x}^{(i)}) \right)^2 \\ &\text{subject to} \quad \sum_{j=0}^d |\theta_j| \leq T \end{aligned}$$

In the **Naive Bayes classifier** we have

- **Type of problem:** classification
- **Model representation:** conditional probability function $P(C|\mathbf{x})$, assuming independence between $\mathbf{x}_1, \dots, \mathbf{x}_d$ (there are no parameters)
- **Cost function:** log-likelihood of the example (maximize):

$$J(\theta, \mathbf{D}) = \log P(C|\mathbf{x})$$

- **Optimization technique:** compute the individual probabilities $P(C)$, $P(C|\mathbf{x}_j)$, $P(\mathbf{x}_j)$ with the data set and apply the Bayes theorem with every new example

Support Vector Machines for Classification

For (1-norm soft margin) **Support Vector Machines for classification** [Cristianini and Shawe-Taylor, 2000]

- **Type of problem:** classification

- **Model representation:**

$$f_{SVM}(\mathbf{x}) = b + \sum_{i=1}^N \alpha_i y^{(i)} K(\mathbf{x}^{(i)}, \mathbf{x}), \text{ where } \boldsymbol{\theta} = (b, \{\alpha_i\}_{i=1}^N)$$

- **Cost function:**

$$\begin{array}{ll} \text{Maximize } \alpha & \sum_{i=1}^N \alpha_i - \frac{1}{2} \sum_{i,j=1}^N \alpha_i \alpha_j y^{(i)} y^{(j)} K(\mathbf{x}^{(i)}, \mathbf{x}^{(j)}) \\ \text{subject to} & \sum_{i=1}^N \alpha_i y^{(i)} = 0 \quad (\text{bias constraint}) \\ & 0 \leq \alpha_i \leq C \quad i = 1 \dots N \end{array}$$

- **Optimization technique:** coordinate descent, gradient projection, decomposition variants (SMO, SMV,...), interior point, primal-dual approaches, etc

In **Bayesian Networks** and other **Probabilistic Graphical Models** we have [Koller and Friedman, 2009]

- **Type of problem:** probability estimation
- **Model representation:** joint probability function $P(\mathbf{x}; \theta)$, with different constraints for different models
- **Cost function:** log-likelihood of the data (maximize):

$$J(\theta, \mathbf{D}) = \sum_{i=1}^N \log P(\mathbf{x}^{(i)}; \theta)$$

- **Optimization technique:** gradient ascent, expectation-maximization, etc

Neural Networks can also be included in this framework:

- **Type of problem:** classification, regression, etc
- **Model representation:** the particular architecture for the problem at hand: Single-layer Perceptron, Multilayer Perceptron, Radial-basis Function Network, Convolutional Neural Network, Recurrent Neural Network, (Restricted) Boltzmann Machine, etc
- **Cost function:** depends on the type of problem (sum-of-squares error, cross-entropy, log-likelihood, etc) with or without regularization
- **Optimization technique:** gradient descent (with many variations), etc

Although these elements are present in all models, sometimes they are difficult to describe in a formal way [Hastie et al., 2001]:

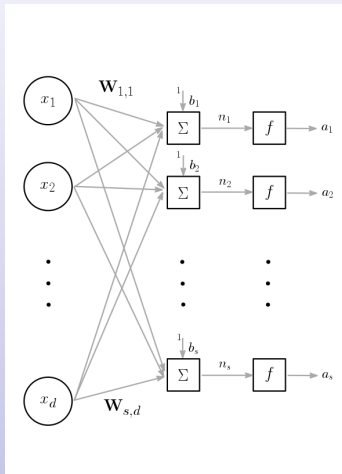
- Sometimes the optimization technique is an heuristic algorithm, as in **Classification Trees**, **Regression Trees**, **Learning Vector Quantization** or **K-Means**, typically based on the optimization of a local property
- Other ones the model representation and the optimization technique are not distinguished, as in the **k-Nearest-Neighbours** classifier

- 1 Introduction
- 2 Linear Regression
- 3 General Machine Learning Issues
- 4 Single-layer Perceptrons**
 - Definition
 - Linear Regression
 - Rosenblatt's Perceptron
 - Logistic Regression
 - Notebook Example Single-layer Perceptrons
- 5 Multilayer Perceptrons

- 1 Introduction
- 2 Linear Regression
- 3 General Machine Learning Issues
- 4 Single-layer Perceptrons**
 - Definition
 - Linear Regression
 - Rosenblatt's Perceptron
 - Logistic Regression
 - Notebook Example Single-layer Perceptrons
- 5 Multilayer Perceptrons

Definition

Single-layer perceptrons are neural networks which only have one layer of weights



- 1 Introduction
- 2 Linear Regression
- 3 General Machine Learning Issues
- 4 Single-layer Perceptrons**
 - Definition
 - **Linear Regression**
 - Rosenblatt's Perceptron
 - Logistic Regression
 - Notebook Example Single-layer Perceptrons
- 5 Multilayer Perceptrons

Although it is not the most common interpretation, the standard version of linear regression (trained with gradient descent), can be seen as a neural networks problem:

- **Type of problem:** regression
- **Model representation:** $h_{\theta}(\mathbf{x}) = \theta_0 + \theta' \cdot \mathbf{x}$
(a single-layer perceptron with linear output units)
- **Cost function:** sum-of-squares error

$$J(\theta, \mathbf{D}) = \frac{1}{2} \sum_{i=1}^N \left(y^{(i)} - h_{\theta}(\mathbf{x}^{(i)}) \right)^2$$

- **Optimization technique:** normal equations, gradient descent with

$$\partial J(\theta_t, \mathbf{D}) / \partial \theta = \mathbf{X}' \mathbf{X} \theta_t - \mathbf{X}' \mathbf{Y} = \mathbf{X}' \left(h_{\theta_t}(\mathbf{X}) - \mathbf{Y} \right)$$

- 1 Introduction
- 2 Linear Regression
- 3 General Machine Learning Issues
- 4 Single-layer Perceptrons**
 - Definition
 - Linear Regression
 - Rosenblatt's Perceptron**
 - Logistic Regression
 - Notebook Example Single-layer Perceptrons
- 5 Multilayer Perceptrons

Rosenblatt's Perceptron

M. Rosenblatt was a psychologist (Neural Networks were born in these fields: W. S. McCulloch was a neurophysiologist, W. H. Pitts was a logician and D. O. Hebb was also a psychologist)

To my knowledge, Rosenblatt's Perceptron [Rosenblatt, 1958] is the first algorithmically described Neural Network, and the first that was implemented in hardware (Mark I Perceptron, 1960)

It is a supervised learning linear model consisting of a single neuron with adjustable synaptic weights and bias

It has several similarities with Support Vector Machines

Rosenblatt's Perceptron



Figure 1: Mark I Perceptron (1960)

Rosenblatt's Perceptron

The Perceptron algorithm [Rosenblatt, 1958]:

Input: (data $\mathbf{D} = \{(\mathbf{x}^{(1)}, y^{(1)}), \dots, (\mathbf{x}^{(N)}, y^{(N)})\}$), $y^{(i)} \in \{-1, +1\}$
 $\boldsymbol{\theta} = 0$ ($\boldsymbol{\theta}$ includes the bias term, with an additional 1 in $\mathbf{x}^{(i)}$)

repeat

for i from 1 to N (for every example):

 Compute the output of $\mathbf{x}^{(i)}$: $h(\mathbf{x}^{(i)}) = \text{sgn}(\boldsymbol{\theta}' \cdot \mathbf{x}^{(i)})$

if $y^{(i)} \cdot h(\mathbf{x}^{(i)}) < 0$ // example bad classified

$\boldsymbol{\theta} = \boldsymbol{\theta} + \eta \cdot y^{(i)} \cdot \mathbf{x}^{(i)}$

until no mistakes are made in the **for** loop

Return $\boldsymbol{\theta}$

Therefore we have:

- **Type of problem:** Classification

We want to assign to a d -dimensional **input vector** $\mathbf{x} = (\mathbf{x}_1, \dots, \mathbf{x}_d)$ its corresponding **class** y

- **Model representation:** $h_{\theta}(\mathbf{x}) = \text{sgn}(\theta' \cdot \mathbf{x}^{(i)})$
- **Cost function:** the number of mistakes (classification accuracy)
- **Optimization technique:** the algorithm previously described

It can be proved that if the patterns (vectors) used to train are linearly separable, then the algorithm converges and its decision surface (an hyperplane) separates the two classes

It could be seen as an algorithm of “pseudo gradient descent” with the objective of minimizing a (non-differentiable) function of the functional margin

$$\sum_{i=1}^N \max \left(0, -y^{(i)} \cdot \boldsymbol{\theta}' \cdot \mathbf{x}^{(i)} \right)$$

- 1 Introduction
- 2 Linear Regression
- 3 General Machine Learning Issues
- 4 Single-layer Perceptrons**
 - Definition
 - Linear Regression
 - Rosenblatt's Perceptron
 - Logistic Regression**
 - Notebook Example Single-layer Perceptrons
- 5 Multilayer Perceptrons

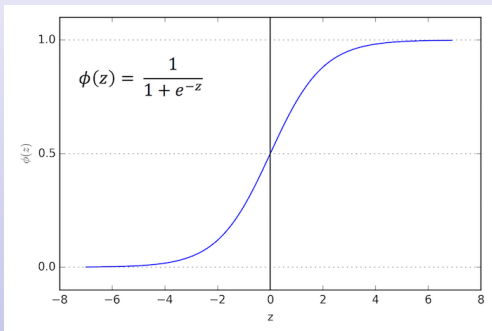
Type of problem: Classification

In order to model it with neural networks, we can assume that (or transform to) $y^{(i)} \in \{0, 1\} \quad i = 1, \dots, N$

Note that, despite its name, **logistic regression refers to a classification problem**

Model representation: a single-layer perceptron with one logistic output unit

- $h_{\theta}(\mathbf{x}) = f(\theta' \cdot \mathbf{x})$
- $f(z) = \frac{1}{1+e^{-z}}$ is the logistic function



Interpretation: The output $h_{\theta}(\mathbf{x})$ is the conditional probability that $\mathbf{x}$ belongs to class 1: $h_{\theta}(\mathbf{x}) = P(y = 1 \mid \mathbf{x}; \theta)$

(linear output units could give values not in $[0, 1]$)

Note that

- The **decision boundary** is linear: $\theta' \cdot \mathbf{x} = 0$
- $\log \left(\frac{P(y=1 \mid \mathbf{x}; \theta)}{P(y=0 \mid \mathbf{x}; \theta)} \right) = \theta' \cdot \mathbf{x}$

Cost function: Cross-entropy

$$J(\theta, D) = \sum_{i=1}^N -y^{(i)} \cdot \log \left(h_{\theta} \left(\mathbf{x}^{(i)} \right) \right) + \\ + \sum_{i=1}^N -(1 - y^{(i)}) \cdot \log \left(1 - h_{\theta} \left(\mathbf{x}^{(i)} \right) \right)$$

which is essentially equivalent to **negative log-likelihood**

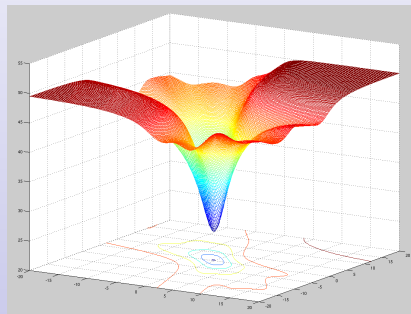
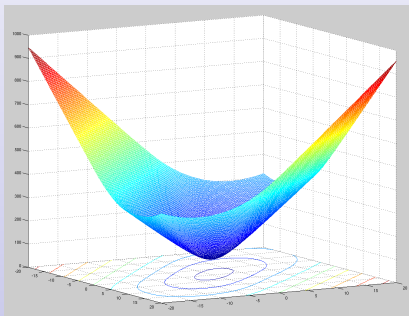
The classification accuracy is not a smooth function!

Interpretation:

- When $y^{(i)} = 0$, we only have $-\log(1 - h_{\theta}(\mathbf{x}^{(i)}))$, that will be small/large if $h_{\theta}(\mathbf{x}^{(i)})$ is near 0/1
- When $y^{(i)} = 1$, we only have $-\log(h_{\theta}(\mathbf{x}^{(i)}))$, that will be small/large if $h_{\theta}(\mathbf{x}^{(i)})$ is near 1/0

Why not using the sum-of-squares error function in this case?

Because **the cross-entropy is convex wrt the parameters**, whereas the sum-of-squares error function is not



Left/Right: cross-entropy/sum-of-squares error function

Optimization technique: gradient descent

The gradient of the cross-entropy error function wrt the parameters **has the same form than the gradient of the sum-of-squares error**

$$\partial J(\theta_t, D) / \partial \theta = \mathbf{X}' \left(h_{\theta_t}(\mathbf{X}) - \mathbf{Y} \right)$$

(you can check it!)

- 1 Introduction
- 2 Linear Regression
- 3 General Machine Learning Issues
- 4 Single-layer Perceptrons**
 - Definition
 - Linear Regression
 - Rosenblatt's Perceptron
 - Logistic Regression
 - **Notebook Example Single-layer Perceptrons**
- 5 Multilayer Perceptrons

Notebook Example Single-layer Perceptrons

Logistic Regression: Run MLPClassifierSklern2D.ipynb with

- Linear models (HSizes = ())
- ActFun = 'logistic'

for

- Two linearly separable classes
- XOR data

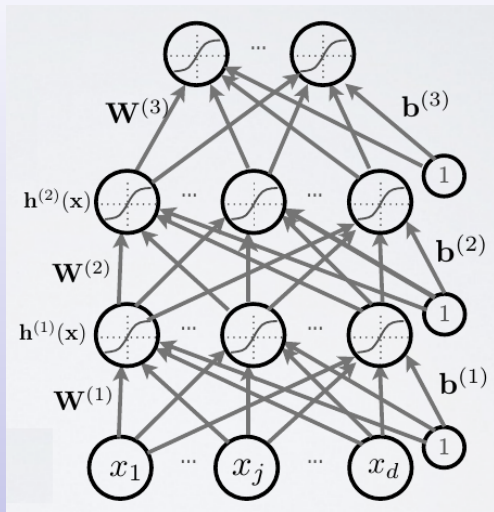
Note that in the example we do not implement the functions, just call them and/or define the structures that will call them (and have been implemented by someone)

- 1 Introduction
- 2 Linear Regression
- 3 General Machine Learning Issues
- 4 Single-layer Perceptrons
- 5 Multilayer Perceptrons**
 - Definition
 - Training MLPs
 - Notebook Example Multilayer Perceptrons
 - Theoretical Results

- 1 Introduction
- 2 Linear Regression
- 3 General Machine Learning Issues
- 4 Single-layer Perceptrons
- 5 Multilayer Perceptrons**
 - **Definition**
 - Training MLPs
 - Notebook Example Multilayer Perceptrons
 - Theoretical Results

Definition

An example of a MLP with 3 layers of weights



Multilayer perceptrons (MLPs) are feed-forward neural networks with several layers of weights, characterized by the fact that the aggregation function is the inner product between the weights and the activation of the previous units

As usual, we will consider the bias terms as standard weights by adding a constant unit of value 1 to the input of every layer

The computation starts in the input layer, and proceeds **layer by layer**: in order to compute the activations of the units a certain layer, **we only need to know the activations of the units of the previous layer** (and the weights of that layer, obviously)

- 1 Introduction
- 2 Linear Regression
- 3 General Machine Learning Issues
- 4 Single-layer Perceptrons
- 5 Multilayer Perceptrons**
 - Definition
 - **Training MLPs**
 - Notebook Example Multilayer Perceptrons
 - Theoretical Results

Training MLPs consists of **finding a set of weights** that optimizes the cost function (the architecture is usually set *a priori*)

For MLPs the most common settings are:

- **Type of problem:** regression, classification
- **Model representation:** the typical architecture of an MLP, where the units are structured by layers
- **Cost function:**
 - Sum-of-squares error for regression problems
 - Cross-entropy for classification problems
 - With or without regularization terms
- **Optimization technique:** based on gradient descent with backpropagation

Backpropagation: Later in this course...

- 1 Introduction
- 2 Linear Regression
- 3 General Machine Learning Issues
- 4 Single-layer Perceptrons
- 5 Multilayer Perceptrons**
 - Definition
 - Training MLPs
 - Notebook Example Multilayer Perceptrons**
 - Theoretical Results

Notebook Example Multilayer Perceptrons

MLPClassifierSklearn2D.ipynb with

- Linear models ($H_{\text{Sizes}} = ()$)
- Non-linear models ($H_{\text{Sizes}} = (N), (N,M), \dots$)

for

- Two linearly separable classes
- XOR data
- Sklearn moons
- Sklearn circles

Again, note that in the example we do not implement the functions, just call them

In particular, the Backpropagation algorithm is used in the function *partial_fit* of *MLPClassifier* with `Solver = 'sgd'`

- 1 Introduction
- 2 Linear Regression
- 3 General Machine Learning Issues
- 4 Single-layer Perceptrons
- 5 Multilayer Perceptrons**
 - Definition
 - Training MLPs
 - Notebook Example Multilayer Perceptrons
 - Theoretical Results**

MLPs with a single layer of hidden units have been shown to be universal approximators for several “well-behaved” classes of functions (including, for example, continuous, integrable or square integrable functions) with many families of activation functions and several metrics [Hornik et al., 1989, Leshno et al., 1993]

That means that we can, in principle, obtain an arbitrarily close approximation to the target function

In general, however, finding a neural network (with any number of hidden units) that models with precision a given training set is an NP-complete problem [Blum and Rivest, 1992, Šíma, 1994]

That's it!

- ▶ Ackley, D. H., Hinton, G. E., and Sejnowski, T. J. (1985). A Learning Algorithm for Boltzmann Machines. *Cognitive Science*, 9:147–169.
- ▶ Amari, S. I. (1967). A theory of Adaptive Pattern Classifiers. *IEEE Transactions on Electronic Computers*, EC16(3):299–307.
- ▶ Amari, S. I. (1972). Learning Patterns and Pattern Sequences by Self-organizing Nets of Threshold Elements. *IEEE Transactions on Computers*, C21:1197–1206.
- ▶ Anderson, J. A. (1972). A Simple Neural Network Generating an Interactive Memory. *Mathematical Biosciences*, 14(3):197–220.
- ▶ Bianco, S., Cadene, R., Celona, L., and Napolitano, P. (2018). Benchmark Analysis of Representative Deep Neural Network Architectures. *IEEE Access*, 6:64270–64277.
- ▶ Blum, A. L. and Rivest, R. L. (1992). Training a 3-node Neural Network is NP-complete. *Neural Networks*, 5(1):117–127.
- ▶ Boser, B. E., Guyon, I. M., and Vapnik, V. N. (1992). A Training Algorithm for Optimal Margin Classifiers. In *5th Annual ACM Workshop on Computational Learning Theory*, pages 144–152.
- ▶ Broomhead, D. S. and Lowe, D. (1988). Radial Basis Functions, Multi-variable Functional Interpolation and Adaptive Networks. Technical Report RSRE-MEMO-4148, Royal Signals and Radar Establishment Malvern (UK).
- ▶ Cortes, C. and Vapnik, V. N. (1995). Support-Vector Networks. *Machine Learning*, 20(3):273–297.
- ▶ Cristianini, N. and Shawe-Taylor, J. (2000). *An Introduction to Support Vector Machines*. Cambridge University Press, UK.
- ▶ Gauss, K. F. (1809). *Theoria Motus Corporum Coelestium in Sectionibus Conicis Solem Ambientium*. Sumtibus F. Perthes et IH Besser.
- ▶ Grossberg, S. (1972). Neural Expectation: Cerebellar and Retinal Analogs of Cells Fired by Learnable or Unlearned Pattern Classes. *Kybernetik*, 10(1):49–57.

Bibliography

- ▶ Grossberg, S. (1980). How Does a Brain Build a Cognitive Code? *Psychological Review*, 87(1):1–51.
- ▶ Hadamard, J. (1908). Mémoire sur le Problème d'Analyse Relatif à l'Équilibre des Plaques Élastiques Encastrées. *Imprimerie Nationale*, 33.
- ▶ Hastie, T., Tibshirani, R., and Friedman, J. H. (2001). *The Elements of Statistical Learning*. Springer-Verlag, New York.
- ▶ Hebb, D. O. (1949). *The Organization of Behavior: A Neuropsychological Theory*. John Wiley & Sons.
- ▶ Hinton, G. E. (1999). Products of Experts. In *International Conference on Artificial Neural Networks*, volume 1, pages 1–6.
- ▶ Hinton, G. E. (2002). Training Products of Experts by Minimizing Contrastive Divergence. *Neural Computation*, 14:1771–1800.
- ▶ Hinton, G. E., Osindero, S., and Teh, Y. (2006). A Fast Learning Algorithm for Deep Belief Nets. *Neural Computation*, 18(7):1527–1554.
- ▶ Hinton, G. E. and Salakhutdinov, R. R. (2006). Reducing the Dimensionality of Data with Neural Networks. *Science*, 313(5786):504–507.
- ▶ Hochreiter, S. and Schmidhuber, J. (1997). Long Short-term Memory. *Neural Computation*, 9(8):1735–1780.
- ▶ Hopfield, J. (1982). Neural Networks and Physical Systems with Emergent Collective Computational Abilities. *Proceedings of the National Academy of Sciences USA*, 79:2554–2558.
- ▶ Hornik, K., Stinchcombe, M., and White, H. (1989). Multilayer Feedforward Networks are Universal Approximators. *Neural Networks*, 2(5):359–366.
- ▶ Horst, R. and Tuy, H. (1993). *Global Optimization: Deterministic Approaches*. Springer-Verlag, Berlin.
- ▶ Ivakhnenko, A. G. (1971). Polynomial Theory of Complex Systems. *IEEE Transactions on Systems, Man and Cybernetics*, 4:364–378.
- ▶ Kohonen, T. (1972). Correlation Matrix Memories. *IEEE Transactions on Computers*, C21(4):353–359.

- ▶ Koller, D. and Friedman, N. (2009). *Probabilistic Graphical Models: Principles and Techniques*. MIT Press.
- ▶ Krizhevsky, A., Sutskever, I., and Hinton, G. E. (2012). ImageNet Classification with Deep Convolutional Neural Networks. In *Advances in Neural Information Processing Systems*, volume 24, pages 1106–1114.
- ▶ Lang, K. J., Waibel, A. H., and Hinton, G. E. (1990). A Time-delay Neural Network Architecture for Isolated Word Recognition. *Neural Networks*, 3(1):23–43.
- ▶ LeCun, Y., Boser, B., Denker, J. S., Henderson, D., Howard, R. E., Hubbard, W., and Jackel, L. D. (1989). Backpropagation Applied to Handwritten Zip Code Recognition. *Neural Computation*, 1(4):541–551.
- ▶ Legendre, A. M. (1805). Nouvelles Méthodes pour la Détermination des Orbites des Comètes. *F. Didot*.
- ▶ Leshno, M., Lin, V. Y., Pinkus, A., and Schocken, S. (1993). Multilayer Feedforward Networks With a Nonpolynomial Activation Function Can Approximate Any Function. *Neural Networks*, 6(6):861–867.
- ▶ McCulloch, W. S. and Pitts, W. (1943). A logical Calculus of the Ideas Immanent in Nervous Activity. *The Bulletin of Mathematical Biophysics*, 5(4):115–133.
- ▶ Minsky, M. L. (1967). *Computation: Finite and Infinite Machines*. Prentice-Hall.
- ▶ Minsky, M. L. and Papert, S. (1969). *Perceptrons: An Introduction to Computational Geometry*. MIT Press.
- ▶ Poggio, T. and Girosi, F. (1990). Networks for Approximation and Learning. *Proceedings of the IEEE*, 78(9):1481–1497.
- ▶ Rosenblatt, F. (1958). The Perceptron: A Probabilistic Model for Information Storage and Organization in the Brain. *Psychological Review*, 65(6):386–408.
- ▶ Rumelhart, D. E., Hinton, G. E., and Williams, R. J. (1986a). Chapter 8: Learning Internal Representations by Error Propagation. In Rumelhart, D. E. and McClelland, J. L., editors, *Parallel Distributed Processing: Explorations in the Microstructure of Cognition (vol. 1)*, pages 318–362. MIT Press.
- ▶ Rumelhart, D. E., Hinton, G. E., and Williams, R. J. (1986b). Learning Internal Representations Back-Propagating Errors. *Nature*, 323:533–536.

Bibliography

- ▶ Rumelhart, D. E. and McClelland, J. L., editors (1986). *Parallel Distributed Processing: Explorations in the Microstructure of Cognition (vols. 1 and 2)*. MIT Press.
- ▶ Smolensky, P. (1986). Chapter 6: Information Processing in Dynamical Systems: Foundations of Harmony Theory. In Rumelhart, D. E. and McClelland, J. L., editors, *Parallel Distributed Processing: Explorations in the Microstructure of Cognition (vol. 1)*, pages 194–281. MIT Press.
- ▶ Šíma, J. (1994). Loading Deep Networks is Hard. *Neural Computation*, 6(5):842–850.
- ▶ Werbos, P. J. (1974). *Beyond Regression: New Tools for Prediction and Analysis in the Behavioral Sciences*. PhD thesis, Harvard University, Boston, MA.
- ▶ Widrow, B. and Hoff, M. E. (1960). Adaptive Switching Circuits. In *IRE WESCON Convention Record*.